Laborator

PDSS

```java
public class Manager {
  public List<String> getPeopleNames(List<Person> people) {
    ArrayList<Person> personList = new ArrayList<>();
    ArrayList<String> stringList = new ArrayList<>();
    for (Person person : people) {
      int x = person.getAge();
      if (x >= 18) {
        personList.add(person);
      }
    }
    for (int i = 0; i < personList.size(); i++) {
      Person p = personList.get(i);
      String s1 = p.getFirstName() + " " + p.getLastName();
      stringList.add(s1);
    }
    for (int i = 0; i < stringList.size(); i++) {
      for (int j = i; j < stringList.size(); j++) {
        if (stringList.get(i).compareToIgnoreCase(stringList.get(j)) > 0) {
          String aux = stringList.get(j);
          stringList.set(j, stringList.get(i));
          stringList.set(i, aux);
        }
      }
    }
    for (String s : stringList) System.out.println(s);
    return stringList;
  }
}
```

```java
public class Person {
  private String firstName;
  private String lastName;
  private int age;
  private Status status;

  //constructor, getters, setters
}
```

```java
public enum Status {
  SINGLE,
  MARRIED,
  DIVORCED,
  WIDOW
}
```

```java
public class PersonService {
    public List<String> getOveragePeopleNames(List<Person> people) {
        List<Person> overagePeople = keepOnlyOveragePeople(people);
        List<String> overagePeopleNames = getPeopleFullNames(overagePeople);
        overagePeopleNames = sortNamesAlphabetically(overagePeopleNames);
        printNames(overagePeopleNames);
        return overagePeopleNames;
    }
...
}
```

```java
private void printNames(List<String> names) {
    for (String name : names)
        System.out.println(name);
}
```

```java
private ArrayList<Person> keepOnlyOveragePeople(List<Person> people) {
    ArrayList<Person> overagePeople = new ArrayList<>();
    for (Person person : people) {
        if (person.getAge() >= 18) {
            overagePeople.add(person);
        }
    }
    return overagePeople;
}
```

```java
private ArrayList<String> getPeopleFullNames
                    (List<Person> overagePeople){
    ArrayList<String> stringList =
                                new ArrayList<>();
    for (Person s : overagePeople) {
        stringList.add(s.getFullName());
    }
    return stringList;   }
```

```java
private List<String> sortNamesAlphabetically(List<String> names) {
    List<String> sortedNames = new ArrayList<>(names);
    for (int i = 0; i < sortedNames.size(); i++)
        for (int j = i; j < sortedNames.size(); j++)
            if (sortedNames.get(i).compareToIgnoreCase(sortedNames.get(j)) > 0)

                String aux = sortedNames.get(j);
                sortedNames.set(j, sortedNames.get(i));
                sortedNames.set(i, aux);
            }
    return sortedNames;
}
```

java.util.Stream<T>

```java
public List<String> getOveragePeopleNames(List<Person> people) {
    return people.stream()
        .filter(person -> person.getAge() >= 18)
        .map(Person::getFullName)
        .sorted()
        .peek(System.out::println)
        .collect(Collectors.toList());
}
```

Streams

Non-terminal Operations

- filter(predicate: Predicate<T>): Stream<T>
- map(mapper: Function<T, R>): Stream<R>
  - mapToInt(mapper: ToIntFunction<T>): IntStream
  - mapToLong(mapper: ToLongFunction<T>): LongStream
  - mapToDouble(mapper: ToDoubleFunction<T>): DoubleStream
- sorted(): Stream<T>
- sorted(comparator: Comparator<T>): Stream<T>
- distinct(): Stream<T>
- peek(consumer: Consumer<T>): Stream<T>
- limit(maxSize: long): Stream<T>
- skip(n: long): Stream<T>

Terminal Operations

- forEach(consumer: Consumer<T>)
- toArray(): Object[]
- collect(collector: Collector)
  - collect(Collectors.toList())
  - collect(Collectors.toSet())
  - collect(Collectors.toMap(.....))
- anyMatch(predicate: Predicate): boolean
- allMatch(predicate: Predicate): boolean
- noneMatch(predicate: Predicate): boolean
- findFirst(): Optional<T>
- findAny(): Optional<T>
- max(comp: Comparator): Optional<T>
- min(comp: Comparator): Optional<T>
- count(): long

- average()
- max()
- min()
- sum()

Exercises

Context

```java
public class Student {
    private String name;
    private String email;
    private int age;
    private String university;
    //constructors, getters, setters
    //toString
    //equals and hashCode by email
}

public class StudentRepository {
    private Collection<Student> students;
    //construtor
}
```

How many students are older than x?

```
public long getNoOfStudentsOlderThan(int age) {

}
```

How many students are older than x?

```
public long getNoOfStudentsOlderThan(int age) {
    return students.stream()
        .filter(student -> student.getAge() > age)
        .count();
}
```

Are all overage students in University?

```
public boolean areAllOverageStudentsInUniversity() {

}
```

Are all overage students in University?

```
public boolean areAllOverageStudentsInUniversity() {
    return students.stream()
        .filter(Student::isOverage)
        .allMatch(student -> student.getUniversity() != null);
}
```

What is the age of the youngest Student?

```
public int getAgeOfYoungestStudent () {



}
```

What is the age of the youngest Student?

```
public int getAgeOfYoungestStudent () {
    return students.stream()          Stream<Student>
        .mapToInt(Student::getAge)    IntStream
        .min()                        OptionalInt
        .getAsInt();
}
```

Which is the oldest Student?

```
public Optional<Student> getOldestStudent() {

}
```

Which is the oldest Student?

```
public Optional<Student> getOldestStudent() {
  return students.stream()
    .max(Comparator.comparing(Student::getAge));
}
```

Get the n-th student ordered by email length

```
public Optional<Student> getNthStudentByEmailLength(int n) {

}
```

Get the n-th student ordered by email length

```
public Optional<Student> getNthStudentByEmailLength(int n) {
    return students.stream()
        .sorted(Comparator.comparingInt(student ->
            student.getEmail().length()))
        .skip(n-1)
        .findFirst();
}
```

Get the average name length of all students

```
public double getAverageNameLengthOfAllStudents() {

}
```

Get the average name length of all students

```
public double getAverageNameLengthOfAllStudents() {
    return students.stream()          Stream<Student>
        .map(Student::getName)        Stream<String>
        .mapToInt(String::length)     IntStream
        .average()                    OptionalDouble
        .orElse(0.);
}
```

Get the list of all students emails

```
public List<String> getAllEmails() {



}
```

Get the list of all students emails

```
public List<String> getAllEmails() {
    return students.stream()
        .map(Student::getEmail)
        .collect(Collectors.toList());
}
```

Stream<Student>

Stream<String>

Get the set of all students names

```
public Set<String> getDistinctNamesSet() {



}
```

Get the set of all students names

```
public Set<String> getDistinctNamesSet() {
    return students.stream()
        .map(Student::getName)
        .collect(Collectors.toSet());
}
```

Stream<Student>

Stream<String>

Get the list of all Universities

```
public List<String> getListOfUniversities() {

}
```

Get the list of all Universities

```
public List<String> getListOfUniversities() {
    return students.stream()
        .map(Student::getUniversity)
        .distinct()
        .collect(Collectors.toList());
}
```

Stream<Student>

Stream<String>

Stream<String>

Get a map with emails to Student

```
public Map<String, Student> getStudentsMappedByEmail() {



}
```

Get a map with emails to Student

```
public Map<String, Student> getStudentsMappedByEmail() {
    return students.stream()
        .collect(Collectors.toMap(
            Student::getEmail,
            Function.identity()))
        );
}
```

Key Generator

Value Generator

Get a map with Universities as keys and all students as values

```
public Map<String, List<Student>> getStudentsGroupedByUniversity() {



}
```

Get a map with Universities as keys and all students as values

```
public Map<String, List<Student>> getStudentsGroupedByUniversity() {
    return students.stream()
        .collect(Collectors.groupingBy(Student::getUniversity));
}
```

Print University, name and age for students in university

```
public void printStudentsInUniversity() {



}
```

Print University, name and age for students in university

```java
public void printStudentsInUniversity() {
  students.stream()
    .filter(student -> student.getUniversity() != null)
    .forEach(student -> System.out.printf(
      "University: %s, Name: %s, Age: %s",
      student.getUniversity(), student.getName(), student.getAge()));
}
```

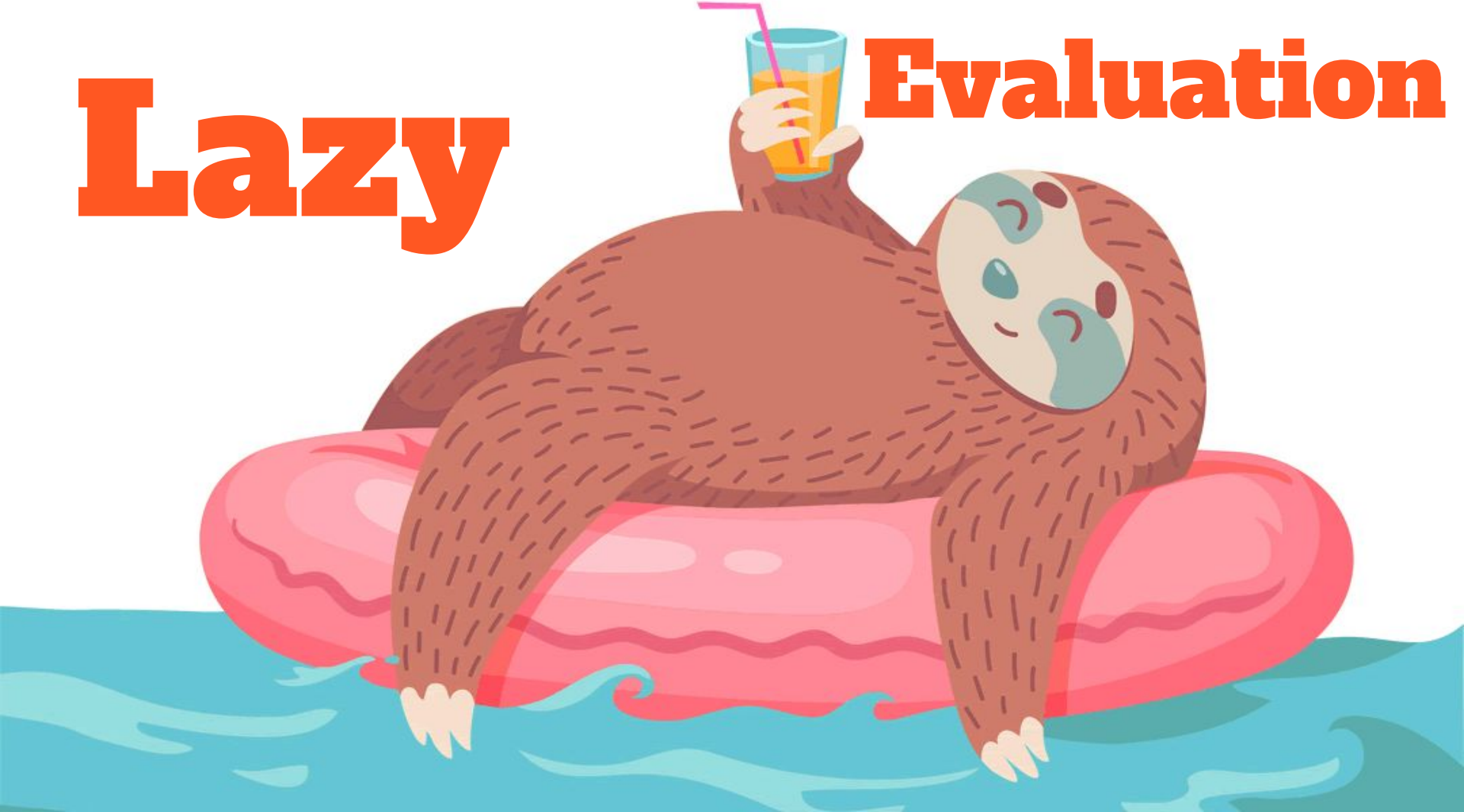
None of the first 5 youngest students have gmail accounts

```
public boolean noneOfTheFiveYoungestStudentsHaveGmail() {



}
```

None of the first 5 youngest students have gmail accounts

```
public boolean noneOfTheFiveYoungestStudentsHaveGmail() {
    return students.stream()
        .sorted(Comparator.comparingInt(Student::getAge))
        .limit(5)
        .noneMatch(student ->
            student.getEmail().endsWith("gmail.com"));
}
```

Lazy Evaluation



Example

```
public boolean allOverageStudentHaveYahooAccounts() {
    return students.stream()
```

Stream<Student>

```
        .filter(Student::isOverage)
```

Stream<Student>

```
        .map(Student::getEmail)
```

Stream<String>

```
        .allMatch(email -> email.endsWith("yahoo.com"));
}
```

References

- Get a Taste of Lambdas and Get Addicted to Streams by Venkat Subramaniam
- Java 8 Lambda Basics by Java Brains
- Introduction to Java 8 Streams by Baeldung
- Java Streams by Baeldung
- Lamdba Expressions and Functional Interfaces: Tips and Best Practices (Chapter 8)